

fracmem

Compressed Fractional Order Memory

Constant time, constant memory filters for fractional order systems

Adithyan Lalu

Guidance, Navigation & Control | Autonomous Systems

August 17, 2026

Overview

1. Motivation and the core challenge
2. Solution approach, in three steps
3. Formulation behind each step
4. Validation: accuracy and performance
5. Deployment targets

Motivation

An ordinary derivative only cares about what just happened.

A **fractional order** derivative is different: it needs to remember *everything* that has ever happened, with old events fading away slowly instead of being forgotten. This shows up in fractional order control systems, aircraft and drone dynamics, materials that slowly deform, and batteries.

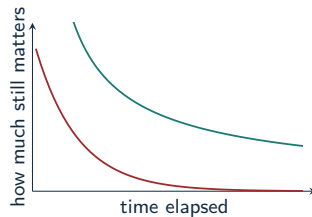
The problem: remembering everything, the naive way, costs a little more compute every single step, forever. On a real microcontroller with fixed memory and a fixed time budget, that eventually breaks.

Key idea. `fracmem` replaces "remember everything" with a small, fixed size trick that never grows, no matter how long it runs.

The Core Challenge

An ordinary system forgets fast. A fractional order system forgets *very* slowly, so cutting its memory short throws away real accuracy.

This slow fading is not a flaw to fix, it is the actual behavior fractional order systems have. The trick is representing it cheaply, not skipping it.



ordinary: forgets fast **fractional order:** forgets slowly

The Governing Equation

First, the pieces: x is the signal you are measuring (say, a sensor reading), x_{k-j} means "the value of x , j steps before now", and α is just a number you pick, like 0.5, that says how "fractional" the derivative is.

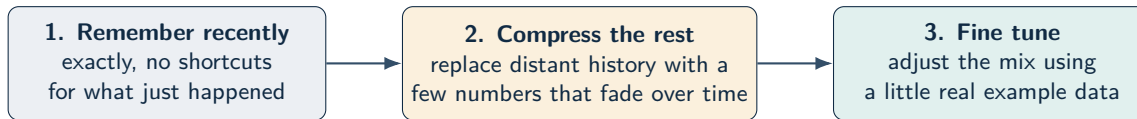
With those pieces, a fractional order derivative is written as:

$$\underbrace{D^\alpha x(t_k)}_{\substack{\text{the answer} \\ \text{we want, right now}}} \approx \underbrace{h^{-\alpha}}_{\substack{\text{a fixed} \\ \text{rescaling number}}} \underbrace{\sum_{j=0}^k w_j x_{k-j}}_{\text{add up the entire past, weighted}}$$

where w_j is one fixed weight per step j , a number that shrinks the further back j points, so recent moments count more than distant ones.

Key idea. The catch: "the entire past" means this sum has more and more terms to add every single step, forever. That growing sum is exactly the problem `fracmem` exists to solve.

Solution Approach



Key idea. Steps 1 and 2 come from pure mathematics, worked out once, ahead of time. Step 3 is the only place real data comes in, just to polish an answer that is already close.

The following slides detail the formulation and validation for each step.

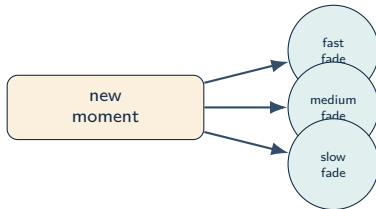
Step 1 | Exact Recent Window



The most recent handful of moments are kept and used exactly as they are, an ordinary, simple running window. No approximation happens here at all. Everything older than that is where the real trick is needed.

Step 2 | Compressed Long Term Memory

Imagine a few small counters. Every new moment tops each counter up a little. Between updates, every counter also fades, a bit like a dimmer light slowly going darker. Add up what is left in all the counters, and that single number stands in for the entire distant past.



Key idea. How fast each counter fades is never guessed or trial and errored: an exact mathematical formula picks the fading speeds, before any real signal is seen.

Step 2 | Formulation

One counter, updated every step by one simple rule:

$$\underbrace{m[k]}_{\text{counter's value, right now}} = \underbrace{\lambda}_{\text{fading speed, a number 0 to 1}} \cdot \underbrace{m[k-1]}_{\text{counter's value, one step ago}} + \underbrace{x[k]}_{\text{newest signal sample}}$$

In words: keep a fraction λ of what the counter already held, then add the newest sample. A small λ fades fast, a λ near 1 fades slowly.

Now use a handful of these counters at once, say p of them, each with its own speed. Label them $m_1[k], m_2[k], \dots$, and give each one a mixing amount $c_1, c_2, \dots$, a number saying how much that counter counts. Add them all up:

$$\underbrace{\hat{T}[k]}_{\text{our stand in for the far past}} = \underbrace{c_1 m_1[k] + c_2 m_2[k] + \dots}_{\text{each counter, times how much it counts}}$$

The speeds $(\lambda_1, \lambda_2, \dots)$ come from an exact formula, decided once. The mixing amounts $(c_1, c_2, \dots)$ are exactly what step 3 figures out next.

Step 3 | Data Driven Calibration

Once the fading speeds are locked in by pure math, one question is left: how much should each counter count?

That is answered with a small amount of real example data, a handful of representative signals, not a huge dataset. Finding the best mix is an easy, always solvable fitting problem, similar to finding the best blend of ingredients from a few taste tests: there is always exactly one best answer, and it can be found directly, with no guessing and no trial and error.

Key idea. Why not also let data choose the fading speeds? Because those already have a provably correct answer from pure math. Only the mixing amounts genuinely benefit from seeing real data.

Step 3 | Formulation

Recall from step 2: the mixing amounts $c_1, c_2, \dots$ are the only unknowns left. Here is how they get found.

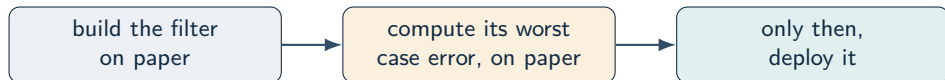
Run the counters on a few example signals where the true answer is already known. For every moment k in those examples, you now have two things: what each counter read, and what the answer should have been:

moment	counter 1	counter 2	counter 3	true answer
$k=1$	0.8	0.3	0.1	0.52
$k=2$	0.6	0.5	0.2	0.48
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

The question becomes: what c_1, c_2, c_3 make $c_1(\text{counter 1}) + c_2(\text{counter 2}) + c_3(\text{counter 3})$ land as close as possible to the true answer, on average, across every row at once? This is called **curve fitting**, one of the oldest, best understood tools in math. It always has exactly one best answer, and a direct formula hands it back instantly, no trial and error, no guessing, no repeated attempts.

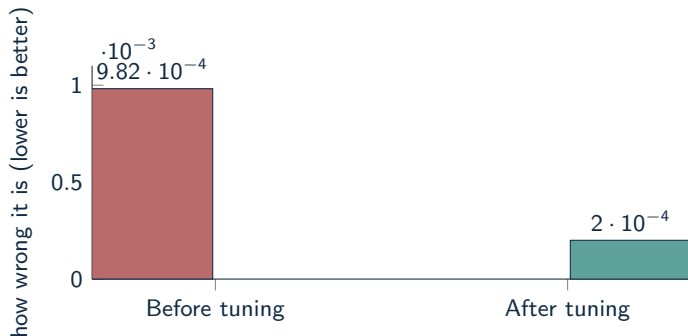
Reliability: An A Priori Error Bound

Before the filter ever runs on a real signal, it is possible to work out, on paper, the worst it could ever possibly be wrong. This is not a promise based on how well it did on a few tests, it is a guarantee, computed in advance, that holds for any input within a known range.



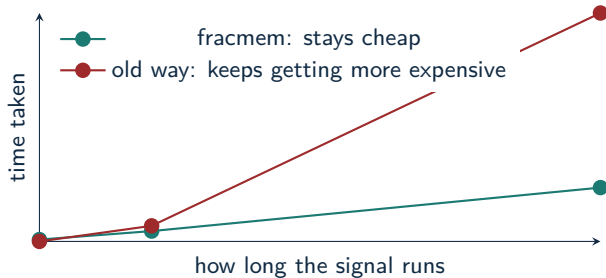
Validation: Accuracy

Comparing the closed form baseline (pure math, zero data) against the calibrated version, fine tuned with a little data:



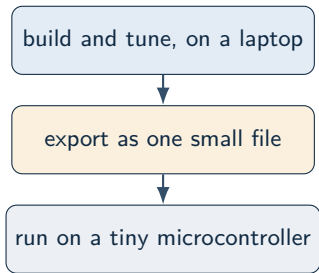
roughly **5 times more accurate** after the data tuning step, for no extra cost to run

Validation: Performance



the longer the signal runs, the bigger fracmem's advantage becomes

Deployment Targets



- Small enough to run on a cheap microcontroller (ESP32), with no extra software installed
- Uses the same, tiny, fixed amount of memory whether it runs for a second or a year
- Useful anywhere fractional order behavior shows up: control systems, materials, batteries

Developer Interface

```
from fracmem import CompressedFractionalFilter

f = CompressedFractionalFilter(alpha=0.5, h=0.01, L=32, p=16)
f.fit(train_signals)          # step 3: tune with a little data
y_hat = f.predict(new_signal) # use it, on a signal of any length
```

four lines: build it, tune it once, and run it

Summary

- Fractional order derivatives need to remember everything, forever, which naively gets more expensive the longer a system runs.
- `fracmem` remembers recent events exactly, and compresses everything older into a few fading counters, chosen by exact math.
- A little real data fine tunes the mix, making it about 5 times more accurate, for free.
- The result is provably safe to deploy, and small enough to run on a two dollar microcontroller.

Thank you

adithyanlalu

Adithyan Lalu | Guidance, Navigation & Control | Autonomous Systems

[linkedin.com/in/adithyan-lalu](https://www.linkedin.com/in/adithyan-lalu)